COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE 010 - Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	July 30, 2026	**DATE SUBMITTED:**	August 1, 2026
NAME:	Garbosa, Isaiah Benedict R.	**INSTRUCTOR:**	Engr. Jimlord Quejado

Hands-on Activity 4.1 Stacks

1. PROCEDURE OUTPUT

ILO A: Implement a stack using the C++ STL (std::stack).

Code:

```cpp
#include <iostream>
#include <stack>

int main(){

    std::stack<int> newStack;

    std::cout<<"is the stack empty? "<< newStack.empty()<<std::endl;

    //inseart a 3 elements
    newStack.push(3);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(2);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
    newStack.push(1);
    std::cout<<"Top of stack: "<< newStack.top()<<std::endl;

    std::cout<<"is the stack empty? "<< newStack.empty()<<std::endl;
    newStack.push(6);
    newStack.push(7);
    newStack.push(8);

    std::cout<<"Size of the stack is: "<< newStack.size()<<std::endl;

```

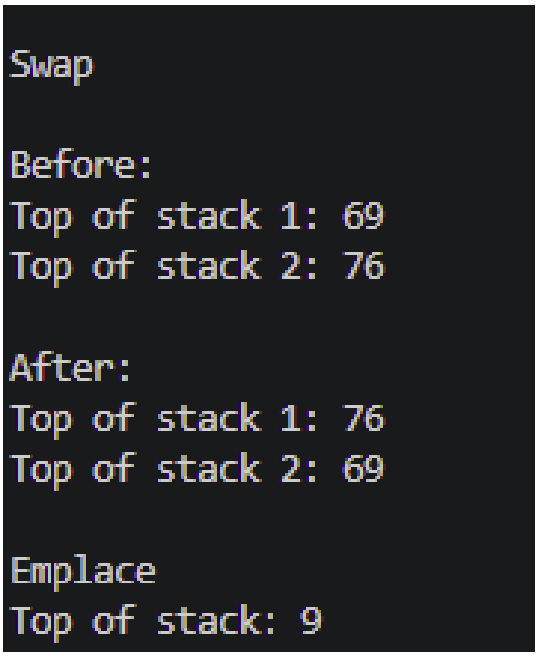
Output

```
is the stack empty? 1
Top of stack: 3
Top of stack: 2
Top of stack: 1
is the stack empty? 0
Size of the stack is: 6
```

Modified Code

```
25      //swap -> swap the content of the public member functions
26
27      std::stack<int> stack1;
28      std::stack<int> stack2;
29
30      std::cout<<"\nSwap\n";
31
32      stack1.push(67);
33      stack1.push(69);
34
35      stack2.push(55);
36      stack2.push(76);
37
38      std::cout<<"\nBefore:"<<std::endl;
39      std::cout<<"Top of stack 1: "<< stack1.top()<<std::endl;
40      std::cout<<"Top of stack 2: "<< stack2.top()<<std::endl;
41
42      stack1.swap(stack2);
43      std::cout<<"\nAfter:"<<std::endl;
44      std::cout<<"Top of stack 1: "<< stack1.top()<<std::endl;
45      std::cout<<"Top of stack 2: "<< stack2.top()<<std::endl;
46
47      //emplace -> construct and insert element -> public member function
48      std::cout<<"\nEmplace\n";
49      newStack.emplace(9);
50      std::cout<<"Top of stack: "<< newStack.top()<<std::endl;
51
52      return 0;
53  }
```

Output



Code Analysis

The program shows the different operations that we can do for stacks using the stack library. It starts by creating a new object with the int data type. We can check if the stack is empty by calling the empty operation, it will return a boolean value. Then, we can check the value of the top value by using the top operation. The size of the stack can be determined using the size operation. After that, there are two more important operations. First is the swap operation which swaps the value of two stacks. For instance, the values of the stack of stack 1 became the value of the stack of stack 2. Finally is the emplace, which acts like the push operations. However, instead of moving the elements, the emplace completely replaces the element.

ILO B.1: Implement a stack using a Static Array.

Initialization

```
1   #include <iostream>
2
3   const size_t maxCap = 100;
4   int stack[maxCap];
5   int top = -1, newData, elements;
6
7   //prototyping
8   bool isEmpty();
9   bool isFull();
10  void push();
11  void pop();
12  void Top();
13  void displayAll();
14
15
16  int main(){
17
18      int choice;
```

Code Analysis
The program starts by setting up important global variables and methods. The variables that were set up are the maxCap which is set to 100, the stack array with the maxCap variable as it's array size, and setting the top as -1, newdata, and elements as int. After that are the prototyping or method declaration for each method that we are going to need.

IsEmpty method

```
bool isEmpty(){
    if(top == -1) return true;
    return false;
}
```

Output

Empty stack:

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
4
1
```

Not empty:

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
4
0
```

Code Analysis

The first method is the isEmpty which checks if the top is empty. This does this by checking if the top is equal to -1 since that is the initial value of the top, so it means that it does not contain anything yet. It returns 1 if true and 0 if false.

isFull method

```
bool isFull(){
    if(top >= elements -1){
        std::cout<<"Stack Overflow";
        return true;
    }
    return false;
}
```

Output

```
Stack operatior

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
5
Stack Overflow1
```

Code Analysis

The isFull method checks if the top is greater then the elements or the number that was set as the max. If it is, it prints out stack overflow

Push method

```
void push(){

    //check if the stack is full, if yes return an error
    if(isFull()) return;
    //ask the user for a value to be pushed into the stack
    std::cout<<"Enter number you want to add to the stack: ";
    std::cin>>newData;
    //store inside the stack
    stack[++top] = newData;
    //
}
```

Output

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
1
Enter number you want to add to the stack: 1
```

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
1
Enter number you want to add to the stack: 2
```

Code Analysis

The push operation checks first if the list is full, then it lets the user input a value that will be the new data. It will be added to the top of the stack.

Pop method

```
void pop(){

    //check if there are no more elements inside the stack
    if(isEmpty()) return;
    //display the element to be popped:
    std::cout<<"Popping: "<<stack[top]<<std::endl;
    //decrement top
    top--;
}
```

Output

Before:

```
Displaying All:

2
1
```

Popping:

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
2
Popping: 2
```

After:

```
Displaying All:

1
```

Code Analysis

The pop method first checks if the list is empty, then it prints out the top of the stack and uses top - - to decrement the top.

Top method

```
void Top(){

    //error checking
    if(isEmpty()) return;

    //display the top element
    std::cout<<"The top element is: "<<stack[top]<<std::endl;
}
```

Output

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
3
The top element is: 1
```

Code Analysis

The top method first checks if the top is empty, then the top element is displayed by calling the top of the stack

DisplayAll method

```
void displayAll(){
    //check if there are no more elements inside
    if(isEmpty()) return;

    std::cout<<"\nDisplaying All: \n"<<std::endl;
    //display
    for(int i = top; i >= 0; i--){
        std::cout<<stack[i]<<std::endl;
    }
}
```

Output

```
Stack operations:

1. PUSH
2. POP
3. Top
4. IsEmpty
5. isFull
6. Display All
6

Displaying All:

2
1
```

Code Analysis

The display all method checks first if the list is empty, then it displays all elements by using a for loop. The loop starts in the top of the stack then it decrements until it reaches zero. It displays the current element for each iteration.

B.2. Stacks using Linked Lists

Initialization

```
#ifndef simply_LL_H
#define simply_LL_H

template <typename T>
class Node{
    public:
    T data;
    Node<T>* next = nullptr;
};

// global
template <typename T>
Node<T>* head = nullptr;
```

Code Analysis

This sets up the node class that contains the public variables of data and the object next. Then, a global object, head, is made.

Push

```
//pushing operation
template <typename T>
void push(T newdata){
    Node<T>* newNode = new Node<T>;
    newNode->data = newdata;
    newNode->next = head<T>;
    head<T> = newNode;
}
```

Code Analysis

The push operation first creates a new node. Then, a new data is assigned as the data for the new node. The next of the node is then set to the head and the head is set to the new node.

isEmpty

```
template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}
```

Code Analysis

The isEmpty checks if the head points to a null pointer, if it does then it means that the stack is empty.

pop

```
template <typename T>
T pop(){
    //if the stack is empty
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow"<<std::endl;
    }
    //saving current top node for temp variable
    Node<T>* temp = head<T>;
    //store the data to a temp varaible
    T tempVal = temp->data;
    //save head to next node
    head<T> = head<T>->next;
    //free up memory
    delete temp;
    //return temp var data
    return tempVal;
}
```

Code Analysis

The pop operation checks first if the stack is empty. Then, if it's not, it sets a temp variable as the head, then a tempVal variable that stores the data of the head. Then, the head is set as the next of that current head, making it the head. Then, delete temp and return tempVal

Top

```
template <typename T>
void Top(){\
    //what if the stack is empty
    if(isEmpty<T>())return;
    std::cout<<"The top is: "<<head<T>->data;
}
```

Code Analysis

The top operation checks first if the stack is empty, if not, it prints out the top by calling the data of the head.

DisplayAll

```
template<typename T>
void DisplayAll(){

    if(isEmpty<T>())return;

    Node<T>* temp = head<T>;
    while(head<T> != nullptr){

        std::cout<<head<T>->data<<" ";
        head<T> = head<T>->next;
    }

    std::cout<<std::endl;

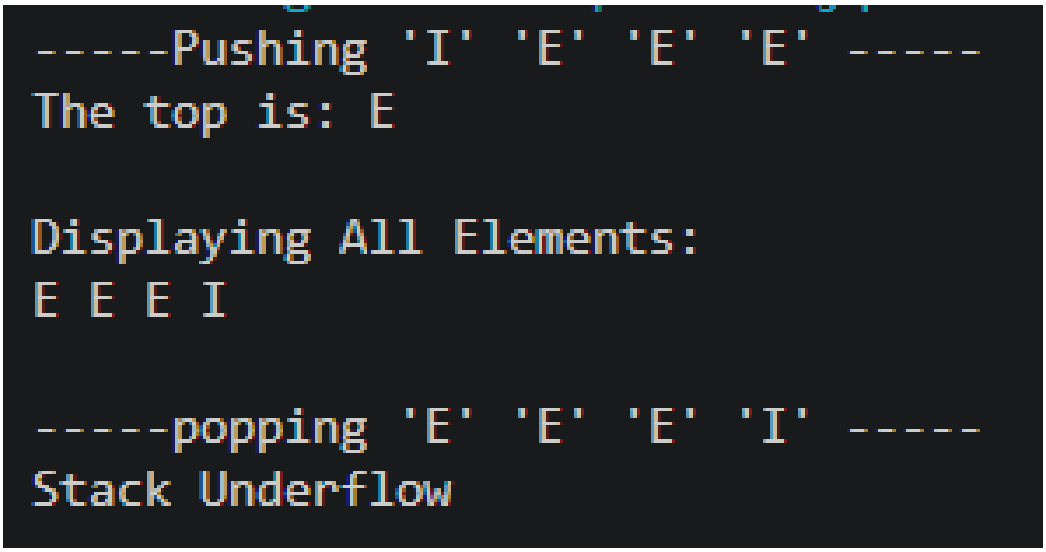

}

#endif
```

Code Analysis

The display all operation first checks if the stack is empty, if not it sets the temp value to the head, then it displays each of the data by printing the data of the head and setting the head to the next node. The loop ends when head is equal to null.

Output


-----Pushing 'I' 'E' 'E' 'E' -----
The top is: E
Displaying All Elements:
E E E I
-----popping 'E' 'E' 'E' 'I' -----
Stack Underflow

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

ILO C: Solve problems using an implementation of stack

stackarray.h

```
#ifndef stackarray_H
#define stackarray_H

#include <string>
#include <iostream>


class stackArray{
private:
    static const size_t maxCap = 100;
    char stack[maxCap];
    int top = -1, elements;
    char newData;
public:
    bool isEmpty(){
        if(top == -1) return true;
        return false;
    }

    void push(){
        stack[++top] = newData;
    }

    char pop() {
            if (top >= 0) {
                return stack[top--];
            }
            return '\0';
        }

    bool checkBalance(std::string expression)
    {
        top = -1;       // reset stack

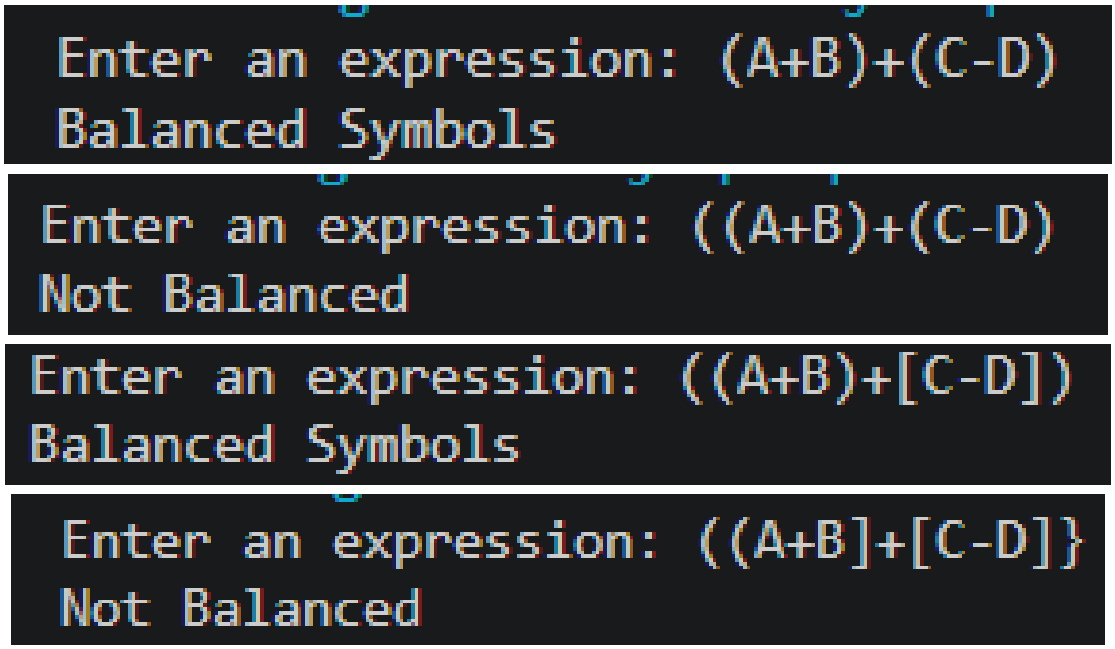
        for (char symbol : expression)
        {
            if (symbol == '(' || symbol == '{' || symbol == '[')
            {
                newData = symbol;
                push();
            }
            else if (symbol == ')' || symbol == '}' || symbol == ']')
            {
                if (isEmpty())
                    return false;
```

```
46
47                  char open = pop();
48
49                  if ((symbol == ')' && open != '(') ||
50                      (symbol == '}' && open != '{') ||
51                      (symbol == ']' && open != '['))
52                  {
53                      return false;
54                  }
55              }
56          }
57
58          return isEmpty();
59      }
60  };
61
62  #endif
```

Mainarray.cpp

```
1   #include <iostream>
2   #include <string>
3   #include "stackarray.h"
4
5   int main(){
6       stackArray stack;
7       std::string expression;
8
9       std::cout << "Enter an expression: ";
10      std::cin>> expression;
11
12      if (stack.checkBalance(expression))
13          std::cout << "Balanced Symbols";
14      else
15          std::cout << "Not Balanced";
16
17      return 0;
18  }
```

Output



Code Analysis

For building the stack with array, we first create a header file. The header file contains a stackArray class with a private class with a const maxCap of 100, a char stack with the maxcap, an int top set to negative one and elements, and a char newData. Then the public class contains the different methods the main program needs. First is the isEmpty which checks if the top is equal to negative one and returns true if it is, a push method that adds a new data to the top of the stack, and a pop method that checks if the top is greater than zero and decrements the top. It returns '\0' since it returns a char, it needs a char also to indicate 0 or nothing. The '/0' means 0 is ASCII. Finally is the main checker, which is called the check balance operation. It starts by setting the top to -1 or empty. A for loop is then used which has the parameter char symbol : expression. This means that it goes through every char symbol in a string statement. Then, it goes to an if statement. If the symbol is an opening symbol, it sets the new data to that symbol and pushes it to the stack. If it is a closing symbol, it sets a new char variable called open which pop the opening symbol that is already in the stack and sets it to open. Then, it compares the closing symbol to that opening symbol, if it matches, then it means that that is a pair, if not, it means that it is not a match. If the stack is empty, then it means that it is balanced, if there is still a symbol in the stack, it returns a false, meaning it's not balanced.

stacklist.h

```cpp
#ifndef stacklist_H
#define stacklist_H

#include <string>

template <typename T>
class Node{
    public:
    T data;
    Node<T>* next = nullptr;
};

// global
template <typename T>
Node<T>* head = nullptr;

//pushing operation
template <typename T>
void push(T newdata){
    Node<T>* newNode = new Node<T>;
    newNode->data = newdata;
    newNode->next = head<T>;
    head<T> = newNode;
}

template <typename T>
bool isEmpty(){
    return head<T> == nullptr;
}

template <typename T>
T pop(){
    //if the stack is empty
    if(isEmpty<T>()){
        std::cout<<"Stack Underflow"<<std::endl;
    }
    //saving current top node for temp variable
    Node<T>* temp = head<T>;
    //store the data to a temp varaible
    T tempVal = temp->data;
    //save head to next node
    head<T> = head<T>->next;
    //free up memory
    delete temp;
    //return temp var data
```

```
46      return tempVal;
47  }
48
49  template <typename T>
50  bool checkBalance(std::string expression)
51  {
52      head<T> = nullptr;      // reset stack
53
54      for (char symbol : expression)
55      {
56          if (symbol == '(' || symbol == '{' || symbol == '[')
57          {
58              push<T>(symbol);
59          }
60          else if (symbol == ')' || symbol == '}' || symbol == ']')
61          {
62              if (isEmpty<T>())
63                  return false;
64
65              char open = pop<T>();
66
67              if ((symbol == ')' && open != '(') ||
68                  (symbol == '}' && open != '{') ||
69                  (symbol == ']' && open != '['))
70              {
71                  return false;
72              }
73          }
74      }
75
76      return isEmpty<T>();
77  }
78
79  #endif
```

Mainlist.cpp

```cpp
#include <iostream>
#include <string>
#include "stacklist.h"

int main()
{
    std::string expression;

    std::cout << "Enter an expression: ";
    std::cin >> expression;

    if (checkBalance<char>(expression))
        std::cout << "Balanced Symbols\n";
    else
        std::cout << "Not Balanced\n";

    return 0;
}
```

Output

```
Enter an expression: (A+B)+(C-D)
Balanced Symbols
```

```
Enter an expression: ((A+B)+(C-D)
Not Balanced
```

```
Enter an expression: ((A+B)+[C-D])
Balanced Symbols
```

```
Enter an expression: ((A+B]+[C-D]}
Not Balanced
```

Code Analysis
The stack linked list functions almost the same as the array. First, we create a header file. We create a class Node with the public class, T data, and we create the node, next that points to a null. We create a global node, head which points to null. The push operation creates a new node, sets the data to new data, sets the next of new node as the current head, and sets the head to the new node. Next is the is empty which checks if the head points to null, making it empty. The pop operation checks first if the stack is empty, then it sets a temp node to the head and the tempval to the temp's data. Then, the head is set to the next of the current head and delete temp. Finally is the check balance which also functions the same as the array. First, we set the head to null. Then, a for loop is used with the parameter char symbol : expression which goes through each character of a string. The if statement checks if the symbol is an opening symbol, if it is, it is pushed to the stack. If it is a closing symbol, it first checks if the stack is empty, if not, then a char open is set to the opening symbol that is popped from the stack. It then compares it to the closing symbol if it is a match. In the end, if the stack is empty, it means that it is balance, if not, it is not balanced.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **N/A**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **N/A**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **N/A**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.